

Model Improvement Review (Lesson - 1)

- Accuracy Score, Precision/Recall / F1 Score

• Example Scenario

- Data set: 1000 people
- Only 10 people have a disease (rare class)

Accuracy Score $\Rightarrow$ Balanced bolishi kerak dataset

• Model A (Bad model)

- Predicts "No Disease" for everyone
- Shuning uchun accuracy'i o'zi yetarli emas va kam
- ~ Precision $\approx$ Prediction (javob) kasal bo'lganda, qanchalik kasalligi to'g'ri
- ~ Recall $\approx$ Qancha kasallik holatlarini model topdi
- ~ F1-Score $\approx$ Precision va Recall o'rtasidagi balance

• F1-Score $\approx \frac{2 \cdot \text{Pre} \cdot \text{Recall}}{\text{Pre} + \text{Recall}}$

- Class imbalance + we need balance between Precision and Recall
- Model: Fraud Detection (Many True positive, false negative)

• Barchasi $\rightarrow$ Juda muhim va complex xollarda

- Self-Driving Car Pedestrian Detection
- High Precision $\rightarrow$ Xaridor xam keraksiz joylarda xam tormoz bosmaydi
- High Recall $\rightarrow$ bironta pijodani e'tiborsiz qoldirmaydi
- Good F1-score $\rightarrow$ balance
- High Accuracy $\rightarrow$ Umumiy ishonarli sistema

• Accuracy score

- Qadon: Balanced data, ^{oqibat} equal cost for wrong positives and wrong negatives
- Model: Predicting if a student will pass an exam (pass/fail)

• Precision

- Qadon: False positives must be minimized $\text{Pre} = \frac{TP}{TP+FP}$
- Model: Email spam or not
- High Precision --- Qadon emailni biz spam desak ko'p xolatlarida to'g'ri dega olamiz

• Recall (to'g'ri xato keb berishi) xat

- False negatives are dangerous
- Chegaradan noqumuniy noradan o'tib ketishiga yo'l qo'yishlik kerak
- High Recall --- Xopis xolatlarini ko'p ushlab olamiz

$$\text{Recall} = \frac{TP}{TP+FN}$$

Model Improvement review (Lesson 2)

- Datalarni to'g'ri va tushunib qoldirilgan qiymatlarni saqlatgan yagona qiymat bilan to'ldirish

Model Improvement review (Lesson 3)

- Good model + bad features = bad predictions

Good features + bad model = bad predictions

Good model + good features = best performance

Feature Selection

- Model ko'p featurelar bilan yaxshi model hosil qiladimi?

- Eng muhim featurelarni tanlab olish modelning yaxshi ishlashini/yaxshi predict qilishini ta'minlaydi (Mantqisizlik) (tezligi train payti)
- O'qat = O'qat kerakli masalliklar.

- Nima uchun foydali?

- Reduces overfitting
- Makes models easier to interpret
- Improves generalization
- Helps remove noisy or redundant feature
- Ko'p masalliq o'qat tanimini buzib qo'yadi
- Speeds up training

M: Uyg'unlikdan oldini

- Qaysi materiallar kerakli
- Qaysilari eng muhim

- Method 1: Correlation Filtering + Mutual Information
- Method 2: ANOVA F-test (Categorical → Continuous target)
- Method 3: Tree Based FI (#Complex xolatlarda yaxshi natija)

Preprocessing va Feature Engineering dan keyin

• Method 1: Correlation Filtering + Mutual Information (Anomaly)

```
from sklearn.feature_selection import mutual_info_classif
import pandas as pd
corr = df.corr()[Target 'Churn'].sort_values(ascending=False)
sortirovka (kattadan kichik)
X = df.drop('Churn', axis=1)
y = df['Churn']
mi = mutual_info_classif(X, y, random_state=42)
mi_series = pd.Series(mi, index=X.columns).sort_values()
print("Correlation Ranking:\n", corr)
print("\n Mutual Information Ranking:\n", mi_series)
```

• Method 2: ANOVA F-test (Categorical → Continuous target)

```
from sklearn.feature_selection import f_classif / f_regression
import pandas as pd
f_values, p_values = f_classif(X, y)
f-value ⇒ katta bo'lsa yaxshi
p-value ⇒ kichik bo'lsa yaxshi
f_df = pd.DataFrame({
    'Feature': X.columns,
    'F_value': f_values,
    'p_value': p_values })
f_df = f_df.sort_values('F_value', ascending=False)
print(f_df)
```

top_k = 10 # threshold g'aymat 10ta eng kamkli
 selected_features = f_df.head(top_k)['Feature'].tolist()
 print(selected_features)

• L1 (Lasso) Regularization # Linear modeller

```
from sklearn.linear_model import Lasso # kırımlı feature'leri oluşturalım
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from pandas as pd
```

```
X = df.drop('Churn', axis=1)
```

```
y = df['Churn']
```

```
lasso = Pipeline([
    ('scaler', StandardScaler()),
    ('model', Lasso(alpha=0.01))])
```

```
lasso.fit(X, y)
```

```
coef = lasso.named_steps['model'].coef_
```

```
lasso_df = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': coef})
```

```
lasso_df['Importance'] = lasso_df['Coefficient'].abs()
```

```
lasso_df = lasso_df.sort_values('Importance', ascending=False)
```

```
print("== Lasso feature Importance (High to low) ==")
```

```
lasso_selected = lasso_df[lasso_df['Importance'] > 0]['Feature'].tolist()
```

```
print("\n Selected Features (Non-zero Coefficients):")
```

```
print(lasso_selected)
```

Lasso giliğundan sonra bu böyükle gələn alqoritmləri fit/traîn gilsək bəddi

Uşuq uşuq X, y ga aqratib dıamiz

```
X = lasso_selecteddan çıqan feature'ları dıb gıamiz'
```

```
y = df['Churn']
```

```
# traîn qaragıni test qaragıni
```

```
# fit qaragıni
```

```
model.fit(X_train, y_train)
```


• Tree Based FI (Classification)

```

from sklearn.ensemble import RandomForestClassifier
import pandas as pd
rf = RandomForestClassifier(n_estimators = 200, random_state = 42)
rf.fit(X, y)
rf_importances = rf.feature_importance_
rf_df = pd.DataFrame({ 'feature': X.columns,
                        'Importance': rf_importances
                      }).sort_values('Importance', ascending = False)
print("=== RANDOM FOREST IMPORTANCE (High → Low) ===")
print(rf_df)

```

Dimensionality Reduction

- # 22ta feature bor desak u ⇒ 22-D deb desak bo'ladi
- # Dimensionality Reduction 22-D ni qisqartirib 10-D ni qilib
 - Multicollinearity ⇒ Bir biriga juda yaqin bo'lgan featureslar
 - 2xil guruh bor faqat bir xildan qamchilash qilmiz
 - PCA = eng muhim bo'lgan featurelarni aniqlamaydi
 - PCA = 2ta bir biriga yaqin bo'lgan featuredan ularni o'rnini bosadigan yangi feature yaratadi
 - Masalan: - B5y va vazndan yangi tana o'lchami degan
- Original features → best combination → Yangi feature → drop original
- 3 xil nevadan alohida alohida ishlatgandan ko'ra ularni blender qilib sharbat qilib ichgan maqsad
- ~ Nima uchun kerak?
 - Dimensionality reduction
 - Noise reduction
 - Better model performance
 - Faster computation

• PCA (Dimensionality Reduction)

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.decomposition import PCA
```

```
import pandas as pd
```

```
import numpy as np
```

```
X = df.drop('Churn', axis=1)
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

```
pca = PCA()
```

```
X_scaled = pca.fit_transform(X_scaled)
```

```
explained_var = pca.explained_variance_ratio_
```

```
pca_df = pd.DataFrame({
    'Principal Component': [f'PC {i+1}' for i in range(len(explained_var))],
    'Explained Variance (%)': explained_var * 100,
    'Cumulative Variance (%)': np.cumsum(explained_var) * 100})
```

```
print("=== PCA Explained Variance Table ===")
```

```
print(pca_df)
```

```
n_components = (np.cumsum(explained_var) < 0.90).sum() + 1
```

```
print(f"Recommended number of components: {n_components}")
```

5. Reduce data using selected components

```
pca_final = PCA(n_components=n_components)
```

```
X_reduced = pca_final.fit_transform(X_scaled)
```

```
print("Shape before PCA:", X.shape)
```

```
print("Shape after PCA:", X_reduced.shape)
```